PetriNetEditorInner - Petri Netエディタ内部コンポーネント

1. 概要

1.1 コンポーネントの目的

PetriNetEditorInnerは、React Flowを使用したペトリネット編集・表示の核となるコンポーネントです。ノード（Place、Transition）とエッジ（Arc）の追加・編集・削除、複数メソッドの管理、テンプレート管理、バリデーション、保存機能など、Petri Netエディタの全機能を統合します。

主要機能:

- ペトリネットの可視化（React Flow）
- ノード・エッジの編集（追加・削除・プロパティ編集）
- 複数メソッド（Method）の管理・切り替え
- テンプレート管理（Material/Condition/Result）
- バリデーション実行・エラー表示
- MAIML保存（S3/ローカルダウンロード）
- Draft機能（自動保存・復元）
- Undo/Redo履歴管理
- ワークフロー追加機能

1.2 使用場面

- PetriNetEditorScreenから呼び出される内部コンポーネント
- 既存MAIMLファイルの編集時
- 新規MAIMLファイルの作成時
- Draft復元時

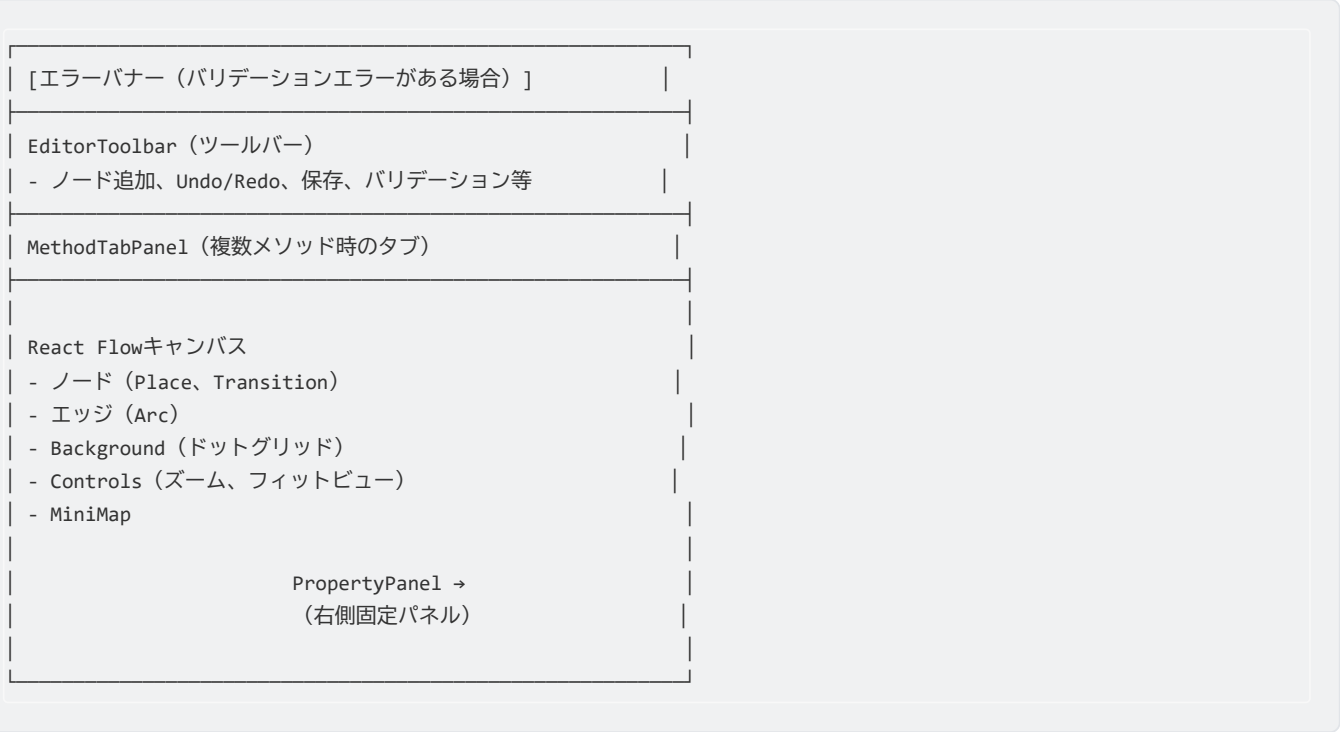
1.3 アクセス権限

- **必須ロール**: Experiment Admin
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造



2.2 主要UI要素

2.2.1 React Flowキャンバス

- **ノードタイプ**:
  - `materialTemplate` - Material Place（黄色、五角形）
  - `conditionTemplate` - Condition Place（青色、五角形）
  - `resultTemplate` - Result Place（紫色、五角形）
  - `transition` - Transition（グレー、四角形）
- **エッジ**: 矢印付き（MarkerType.ArrowClosed）
- **Background**: ドットグリッド（16px間隔）
- **Controls**: ズーム、フィットビュー
- **MiniMap**: 右下に表示

2.2.2 PropertyPanel（右側パネル）

- ノード選択時: ノードプロパティ編集
- エッジ選択時: エッジID編集
- テンプレート編集ボタン（Placeノードのみ）

2.2.3 ダイアログ類

- `SaveConfirmDialog` - 保存確認・バリデーション結果表示
- `TemplateEditorDialog` - テンプレート編集
- `DocumentEditDialog` - Document情報編集
- `ValidationPanel` - バリデーション結果詳細・自動補正
- `WorkflowSelectorDialog` - ワークフロー追加
- `DraftManager` - Draft管理

2.3 ビューモード

2.3.1 petriNetモード（デフォルト）

- React Flowキャンバスを表示
- ノード・エッジの編集が可能

2.3.2 templatesモード

- TemplateList表示
- 共通テンプレート/メソッド内テンプレートの切り替え
- テンプレート追加・削除・編集

---

3. 機能仕様

3.1 Props定義

```
export interface PetriNetEditorInnerProps {
  maimlData: string | null;
  onError?: (error: Error) => void;
  editingFileName?: string;
  editMode: string;
  /** DocumentData.id の初期値（新規作成時のみ適用） */
  initialDocumentId?: string;
  /** DocumentData.description の初期値（新規作成時のみ適用） */
  initialDescription?: string;
  originalFileName?: string;
  sourceS3Key?: string;
```

```
  [key: string]: unknown;
}
```

Props説明:

Prop名	型	必須	説明
maimlData	string | null	✅	MAIML XMLデータ（nullの場合は新規作成モード）
onError	(error: Error) => void	-	エラー発生時のコールバック
editingFileName	string	-	編集中のファイル名
editMode	string	-	編集モード（'project', 'format', 'draft'）
initialDocumentId	string	-	新規作成時のDocument ID初期値
initialDescription	string	-	新規作成時のDocument説明初期値
originalFileName	string	-	元のファイル名（保存時に使用）
sourceS3Key	string	-	元のS3キー（保存先決定に使用）

3.2 State定義

```
// React Flowノード・エッジ
const [nodes, setNodes, onNodesChange] = useNodesState<NodeData>([]);
const [edges, setEdges, onEdgesChange] = useEdgesState<EdgeData>([]);

// ツール選択
const [selectedTool, setSelectedTool] = useState<'select' | 'workflow'>('select');

// 選択状態
const [selectedNode, setSelectedNode] = useState<Node<NodeData> | null>(null);
const [selectedEdge, setSelectedEdge] = useState<Edge<EdgeData> | null>(null);
const [selectedNodes, setSelectedNodes] = useState<Node<NodeData>[]>([]);

// クリップボード
const [clipboard, setClipboard] = useState<ClipboardData | null>(null);
const [pressedKeys, setPressedKeys] = useState<Set<string>>(new Set());

// ダイアログ開閉
const [saveConfirmDialogOpen, setSaveConfirmDialogOpen] = useState<boolean>(false);
const [templateEditorOpen, setTemplateEditorOpen] = useState<boolean>(false);
const [documentDialogOpen, setDocumentDialogOpen] = useState<boolean>(false);
const [workflowDialogOpen, setWorkflowDialogOpen] = useState<boolean>(false);
const [draftManagerOpen, setDraftManagerOpen] = useState<boolean>(false);
const [validationPanelOpen, setValidationPanelOpen] = useState<boolean>(false);

// バリデーション
const [validationResult, setValidationResult] = useState<AnalysisResult | null>(null);
const [criticalErrors, setCriticalErrors] = useState<ErrorContent[]>([]);
const [nonCriticalErrors, setNonCriticalErrors] = useState<ErrorContent[]>([]);

// 状態フラグ
```

```
const [isLoading, setIsLoading] = useState<boolean>(false);
const [isValidating, setIsValidating] = useState<boolean>(false);
const [isCorrecting, setIsCorrecting] = useState<boolean>(false);
const [isSaving, setIsSaving] = useState<boolean>(false);

// エラー
const [error, setError] = useState<string | null>(null);

// テンプレート編集
const [templateEditNode, setTemplateEditNode] = useState<Node<NodeData> | null>(null);

// Document情報
const [documentData, setDocumentData] = useState<DocumentData | null>(null);

// ワークフロー追加
const [availableFiles, setAvailableFiles] = useState<Array<{ key: string; name: string; lastModified?:
string; size?: number }>>([]);
const [isLoadingFiles, setIsLoadingFiles] = useState<boolean>(false);
const [pendingClickPosition, setPendingClickPosition] = useState<{ x: number; y: number } | null>(null);

// Draft管理
const [draftModeHandled, setDraftModeHandled] = useState<boolean>(false);

// ビューモード
const [viewMode, setViewMode] = useState<'petriNet' | 'templates'>('petriNet');
const [selectedTemplate, setSelectedTemplate] = useState<Template | null>(null);
const [showCommonTemplates, setShowCommonTemplates] = useState<boolean>(false);
const [savedPetriNetViewport, setSavedPetriNetViewport] = useState<{ x: number; y: number; zoom: number } |
null>(null);

// 複数Method管理
const [methods, setMethods] = useState<MethodData[]>([]);
const [activeMethodIndex, setActiveMethodIndex] = useState<number>(0);
const [multiMethodMode, setMultiMethodMode] = useState<boolean>(false);
```

主要Stateの説明:

State名	型	初期値	説明
nodes	Node[]	[]	React Flowノード配列
edges	Edge[]	[]	React Flowエッジ配列
selectedNode	Node | null	null	現在選択中のノード
validationResult	AnalysisResult | null	null	バリデーション結果
methods	MethodData[]	[]	複数メソッドの配列
activeMethodIndex	number	0	アクティブなメソッドのインデックス
documentData	DocumentData | null	null	Document情報（ID、名前、説明等）
viewMode	'petriNet' | 'templates'	'petriNet'	表示モード

3.3 カスタムフック使用

```
// React Flow
const { fitView, screenToFlowPosition, getViewport, setViewport } = useReactFlow();

// 履歴管理（Undo/Redo）
const { history, undo, redo, canUndo, canRedo, pushState, reset } = useHistory({
  nodes: [],
  edges: [],
});

// ワークフロー追加
const { addWorkflow } = useWorkflow({
  addToHistory: (action: string, data: unknown) => { /* ... */ },
});

// Draft管理
const {
  autoSaveEnabled,
  setAutoSaveEnabled,
  lastSavedDraft,
  lastAutoSaveTime,
} = useDraftManager(currentEditState, {
  userId: currentUserId,
  autoSaveInterval: 30 * 1000, // 30秒
  onAutoSaveSuccess: (draft) => { /* ... */ },
  onAutoSaveError: (error) => { /* ... */ },
});

// キーボードショートカット
const { pressedKeys: keysPressedFromHook } = useKeyboardShortcuts({
  nodes,
  clipboard,
  canUndo,
  canRedo,
  selectedNodes,
  templateEditorOpen,
  saveConfirmDialogOpen,
  workflowDialogOpen,
  undo,
  redo,
  handleCopy,
  handlePaste,
  handleOpenSaveConfirmDialog,
});

// エディタアクション
const { handleCopy, handlePaste } = useEditorActions({
  nodes,
  edges,
  selectedNodes,
  clipboard,
  setNodes,
  setEdges,
  setClipboard,
  setSelectedNode,
  setSelectedEdge,
  pushState,
```

```
});

// テンプレート管理（TemplateProvider経由）
const { templateErrors, addTemplate, deleteTemplate, updateTemplate, allTemplates } = useTemplates();
```

---

4. データ型定義

4.1 NodeData型

```
interface NodeData {
  id: string;
  name: string;
  description: string;
  placeType: string;
  nodeType: NodeType; // 'place' | 'transition'
  template?: Template;
  templateRef?: Template;
  isCommonTemplate?: boolean;
  methodId?: string | null;
  commonTemplateRef?: string;
  hasError?: boolean;
  errors?: ErrorContent[];
}
```

4.2 MethodData型

```
interface MethodData {
  methodId: string;
  methodIndex: number;
  name: string;
  nodes: Node<NodeData>[];
  edges: Edge<EdgeData>[];
  viewport: { x: number; y: number; zoom: number };
}
```

4.3 DocumentData型

```
interface DocumentData {
  id: string;
  name: string;
  description?: string;
```

```
  methodId: string;
  // その他のフィールド
}
```

---

5. 主要機能

5.1 初期化・データ読み込み

5.1.1 新規作成モード

```
// maimlData が null の場合
if (!maimlData) {
  const { documentData: initialDocumentData } = generateInitialDocumentData(editingFileName, {
    documentId: initialDocumentId,
    description: initialDescription,
  });
  setDocumentData(initialDocumentData);

  const initialMethod = generateInitialMethod(initialDocumentData.methodId);
  setMethods([initialMethod]);

  setNodes([]);
  setEdges([]);
  reset({ nodes: [], edges: [] });
}
```

5.1.2 既存MAIMLファイル読み込み

```
// maimlParserTypedでパース
const parser = new MaimlParserTyped(maimlData);
const methodsData = parser.getMethods();

// 複数メソッドの場合
if (methodsData.length > 1) {
  setMultiMethodMode(true);
}

// 各methodをReact Flow形式に変換
const convertedMethods = methodsData.map((method: Method, index: number): MethodData => {
  const { nodes: methodNodes, edges: methodEdges } =
    convertToReactFlowFormat(parser, methodPetriNetData, templateMap);

  return {
    methodId: method.id,
    methodIndex: index,
```

```
    name: `Method ${index + 1}`,
    nodes: methodNodes,
    edges: methodEdges,
    viewport: { x: 0, y: 0, zoom: 1 }
  };
});

setMethods(convertedMethods);
setActiveMethodIndex(0);
setNodes(convertedMethods[0].nodes);
setEdges(convertedMethods[0].edges);
```

5.1.3 自動バリデーション実行

```
// 読み込み完了後、500ms後に自動バリデーション実行
setTimeout(async () => {
  const base64Data = btoa(unescape(encodeURIComponent(maimlData)));
  const validateResponse = await maimlService.validateMaiml(base64Data);

  if (validateResponse.success) {
    const validateResult = validateResponse.data as AnalysisResult;
    setValidationResult(validateResult);
  }
}, 500);
```

5.2 ノード・エッジ操作

5.2.1 ノード追加

```
const handleAddNode = useCallback((nodeType: string) => {
  const newNode: Node<NodeData> = {
    id: `${nodeType}-${Date.now()}`,
    type: nodeType === 'transition' ? 'transition' :
          nodeType === 'material' ? 'materialTemplate' :
          nodeType === 'condition' ? 'conditionTemplate' : 'resultTemplate',
    position: {
      x: Math.random() * 400 + 100,
      y: Math.random() * 400 + 100,
    },
    data: {
      id: `${nodeType}-${Date.now()}`,
      name: '',
      description: '',
      placeType: nodeType,
      nodeType: (nodeType === 'transition' ? 'transition' : 'place') as NodeType,
    },
  };
```

```
  setNodes((nds) => {
    const newNodes = [...nds, newNode];
    pushState({ nodes: newNodes, edges });
    return newNodes;
  });
}, [setNodes, edges, pushState]);
```

5.2.2 ノードプロパティ更新

```
const handleUpdateNodeProperty = useCallback((nodeId: string, newData: NodeData) => {
  setNodes((nds) => {
    const newNodes = nds.map((node) =>
      node.id === nodeId ? { ...node, data: newData } : node
    );
    pushState({ nodes: newNodes, edges });

    const updatedNode = newNodes.find(n => n.id === nodeId);
    if (updatedNode) {
      setSelectedNode(updatedNode);
    }

    return newNodes;
  });
}, [setNodes, edges, pushState]);
```

5.2.3 ノードID変更

```
const handleUpdateNodeId = useCallback((oldId: string, newId: string) => {
  // TemplateContextのテンプレートも更新（placeRefsの参照を更新）
  if (window.clineAllTemplates && window.clineUpdateTemplate) {
    // テンプレートのplaceRefsを更新
  }

  setNodes((prevNodes) => {
    const newNodes = prevNodes.map((node) =>
      node.id === oldId
        ? { ...node, id: newId, data: { ...node.data, id: newId } }
        : node
    );

    setEdges((prevEdges) => {
      const newEdges = prevEdges.map((edge) => ({
        ...edge,
        source: edge.source === oldId ? newId : edge.source,
        target: edge.target === oldId ? newId : edge.target,
      }));

      pushState({ nodes: newNodes, edges: newEdges });
      return newEdges;
    });
```

```
    return newNodes;
  });
}, [setNodes, setEdges, pushState]);
```

5.2.4 ノード削除

```
const handleNodesDelete = useCallback((deletedNodes: Node<NodeData>[]) => {
  const deletedIds = deletedNodes.map(n => n.id);

  // 削除されるノードに接続されているエッジを特定
  const affectedEdges = edges.filter((edge: Edge<EdgeData>) =>
    deletedIds.includes(edge.source) ||
    deletedIds.includes(edge.target)
  );

  // エッジを明示的に削除
  setEdges((eds: Edge<EdgeData>[]) => eds.filter((e: Edge<EdgeData>) =>
    !affectedEdges.find((ae: Edge<EdgeData>) => ae.id === e.id)
  ));

  // ノードを削除
  setNodes((nds: Node<NodeData>[]) => {
    const newNodes = nds.filter((n: Node<NodeData>) => !deletedIds.includes(n.id));
    const newEdges = edges.filter((e: Edge<EdgeData>): boolean => !affectedEdges.find((ae: Edge<EdgeData>) =>
ae.id === e.id));
    pushState({ nodes: newNodes, edges: newEdges });
    return newNodes;
  });
}, [edges, setEdges, setNodes, pushState]);
```

5.2.5 エッジ接続

```
const handleConnect = useCallback((connection: Connection) => {
  const edgeId = `arc-${Date.now()}`;
  const newEdge = {
    ...connection,
    id: edgeId,
    type: 'default',
    animated: false,
    style: { strokeWidth: 2 },
    markerEnd: {
      type: MarkerType.ArrowClosed,
      color: '#333',
      width: 20,
      height: 20,
    },
    data: {
      id: edgeId,
    }
```

```
  };

  setEdges((eds) => {
    const newEdges = addEdge(newEdge, eds);
    pushState({ nodes, edges: newEdges });
    return newEdges;
  });
}, [setEdges, nodes, pushState]);
```

5.3 複数メソッド管理

5.3.1 メソッド切り替え

```
const handleMethodChange = useCallback((newIndex: number) => {
  // 1. 現在のmethod状態を保存（ビューポート含む）
  const currentViewport = getViewport();
  setMethods(prev => {
    const updated = [...prev];
    updated[activeMethodIndex] = {
      ...updated[activeMethodIndex],
      nodes: nodes,
      edges: edges,
      viewport: currentViewport
    };
    return updated;
  });

  // 2. 新しいmethodのノード/エッジを読み込み
  const newMethod = methods[newIndex];
  setNodes(newMethod.nodes);
  setEdges(newMethod.edges);
  setActiveMethodIndex(newIndex);

  // 3. ビューポート復元
  if (newMethod.viewport) {
    setTimeout(() => {
      setViewport(newMethod.viewport);
    }, 50);
  }
}, [methods, activeMethodIndex, nodes, edges, setNodes, setEdges, getViewport, setViewport]);
```

5.3.2 メソッド追加

```
const handleAddMethod = useCallback(() => {
  const timestamp = Date.now();
  const newMethod = {
    methodId: `method_${timestamp}`,
    methodIndex: methods.length,
```

```
    name: `Method ${methods.length + 1}`,
    nodes: [],
    edges: [],
    viewport: { x: 0, y: 0, zoom: 1 }
  };

  const currentViewport = getViewport();
  setMethods(prev => {
    const updated = [...prev];
    updated[activeMethodIndex] = {
      ...updated[activeMethodIndex],
      nodes: nodes,
      edges: edges,
      viewport: currentViewport
    };
    return [...updated, newMethod];
  });

  setNodes([]);
  setEdges([]);
  setActiveMethodIndex(methods.length);
  setMultiMethodMode(true);
}, [methods, activeMethodIndex, nodes, edges, getViewport, setNodes, setEdges]);
```

---

6. テンプレート管理

6.1 テンプレート編集ダイアログを開く

```
const handleOpenTemplateEditor = useCallback((node: Node<NodeData>) => {
  const isCommon = node.data.isCommonTemplate || false;
  const currentMethodId = methods[activeMethodIndex]?.methodId;
  const nodeWithMethodId: Node<NodeData> = {
    ...node,
    data: {
      ...node.data,
      methodId: isCommon
        ? null
        : (node.data.methodId ?? currentMethodId ?? null),
    },
  };

  setTemplateEditNode(nodeWithMethodId);
  setTemplateEditorOpen(true);
}, [methods, activeMethodIndex]);
```

6.2 テンプレート保存

```
const handleSaveTemplate = useCallback((
  nodeId: string,
  templateRef: Template,
  isNewTemplate: boolean,
  templateCategory: string,
  isCommonTemplate: boolean,
  methodId: string | undefined
) => {
  const templateId = templateRef.id || nodeId;

  // propertiesを配列形式に変換
  const convertedTemplateRef = {
    ...templateRef,
    properties: templateRef.properties && typeof templateRef.properties === 'object' &&
!Array.isArray(templateRef.properties)
      ? convertNestedPropertiesToArray(templateRef.properties)
      : templateRef.properties || []
  };

  if (isNewTemplate) {
    // 新規テンプレート作成
    const correctMethodId = isCommonTemplate ? undefined : (methodId || documentData?.methodId ||
methods[activeMethodIndex]?.methodId);

    const newTemplateForContext: Template = {
      id: templateId,
      uuid: convertedTemplateRef.uuid,
      name: convertedTemplateRef.name || '',
      description: convertedTemplateRef.description,
      category: templateCategory,
      properties: convertedTemplateRef.properties || [],
      contents: convertedTemplateRef.contents || [],
      placeRefs: convertedTemplateRef.placeRefs || [],
      templateRef: convertedTemplateRef.templateRef,
      methodId: correctMethodId,
    };

    if (window.clineAddTemplate) {
      window.clineAddTemplate({
        category: templateCategory as 'material' | 'condition' | 'result',
        template: newTemplateForContext,
      });
    }

    // nodesのdata.templateを更新
    setNodes((nds) => {
      const placeRefsToUpdate = newTemplateForContext.placeRefs || [];
      const newNodes = nds.map((node: Node<NodeData>) => {
        if (placeRefsToUpdate.some((pr) => pr.ref === node.id)) {
          return {
            ...node,
            data: {
              ...node.data,
              template: newTemplateForContext,
              templateRef: newTemplateForContext,
              isCommonTemplate: isCommonTemplate,
```

```
            },
          };
        }
        return node;
      });
      pushState({ nodes: newNodes, edges });
      return newNodes;
    });
  } else {
    // 既存テンプレート更新
    if (window.clineUpdateTemplate) {
      window.clineUpdateTemplate(templateId, convertedTemplateRef);
    }

    // nodesのdata.templateを更新
    setNodes((nds) => {
      const placeRefsToUpdate = convertedTemplateRef.placeRefs || [];
      const newNodes = nds.map((node: Node<NodeData>) => {
        if (placeRefsToUpdate.some((pr) => pr.ref === node.id)) {
          return {
            ...node,
            data: {
              ...node.data,
              template: convertedTemplateRef,
              templateRef: convertedTemplateRef,
              isCommonTemplate: isCommonTemplate,
            },
          };
        }
        return node;
      });
      pushState({ nodes: newNodes, edges });
      return newNodes;
    });
  }
}, [setNodes, edges, pushState, convertNestedPropertiesToArray]);
```

---

7. バリデーション・保存機能

7.1 保存確認ダイアログを開く

```
const handleOpenSaveConfirmDialog = useCallback(async () => {
  try {
    // MaiML生成
    const metadata = {
      name: documentData?.name || 'document',
      description: documentData?.description || '',
    };

    const allTemplatesFromContext = window.clineAllTemplates || null;
```

```
    let maimlXml;
    if (multiMethodMode && methods.length > 0) {
      // 複数メソッド
      maimlXml = convertMultipleMethodsToMaiML(updatedMethods, metadata, documentData,
allTemplatesFromContext);
    } else {
      // 単一メソッド
      maimlXml = await convertReactFlowToMaiML(nodes, edges, metadata, documentData, null,
allTemplatesFromContext);
    }

    // Base64エンコード
    const base64Data = btoa(unescape(encodeURIComponent(maimlXml)));

    // バリデーションAPI呼び出し
    const response = await maimlService.validateMaiml(base64Data);

    if (!response.success) {
      throw new Error(`バリデーションエラー: ${response.error?.message}`);
    }

    const result = response.data as AnalysisResult;
    setValidationResult(result);

    // SaveConfirmDialogを開く
    setSaveConfirmDialogOpen(true);

  } catch (err: unknown) {
    const errorMessage = getErrorMessage(err);
    alert(`バリデーション実行に失敗しました: ${errorMessage}`);
  }
}, [nodes, edges, documentData, getViewport, methods, activeMethodIndex, multiMethodMode]);
```

7.2 保存実行

```
const handleSaveConfirmAction = useCallback(async (action: string, saveDestination: string, folder?: string)
=> {
  if (action === 'cancel') {
    handleCloseSaveConfirmDialog();
    return;
  }

  try {
    setIsSaving(true);

    // MaiML生成
    const metadata = {
      name: documentData?.name || 'document',
      description: documentData?.description || '',
    };

    let maimlXml;
```

```
    if (multiMethodMode && methods.length > 0) {
      maimlXml = convertMultipleMethodsToMaiML(updatedMethods, metadata, documentData,
allTemplatesFromContext);
    } else {
      maimlXml = await convertReactFlowToMaiML(nodes, edges, metadata, documentData ?? null, null,
allTemplatesFromContext);
    }

    // 自動補正の場合
    if (action === 'auto-correct') {
      const base64Data = btoa(unescape(encodeURIComponent(maimlXml)));
      const response = await maimlService.autoCorrectMaimlRealtime(base64Data, [], currentOrgId);

      if (response.success && response.data.correctedContent) {
        maimlXml = atob(response.data.correctedContent);
      }
    }

    // 保存先に応じて処理を分岐
    if (saveDestination === 's3') {
      // S3保存
      const fileName = `${baseFileName}.maiml`;
      const fileKey = folder ? `${folder}/${fileName}` : `${currentOrgId}/${fileName}`;

      const base64Data = btoa(unescape(encodeURIComponent(maimlXml)));
      const archive = editMode === 'project';

      const response = await maimlService.saveMaiml(fileKey, base64Data, archive);

      if (!response.success) {
        throw new Error(response.error?.message || 'S3保存に失敗しました');
      }

      alert(`ペトリネット "${metadata.name}" をS3に保存しました`);
    } else {
      // ローカルダウンロード
      const blob = new Blob([maimlXml], { type: 'application/xml' });
      const url = URL.createObjectURL(blob);

      const link = document.createElement('a');
      link.href = url;
      link.download = `${fileName}.maiml`;
      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
      URL.revokeObjectURL(url);

      alert(`ペトリネット "${fileName}" をダウンロードしました`);
    }

    setSaveConfirmDialogOpen(false);
    setValidationResult(null);

  } catch (err: unknown) {
    const errorMessage = getErrorMessage(err);
    alert(`保存に失敗しました: ${errorMessage}`);
  } finally {
    setIsSaving(false);
```

```
  }
}, [nodes, edges, documentData, handleCloseSaveConfirmDialog]);
```

---

8. キーボードショートカット

8.1 useKeyboardShortcuts

```
// M: Material Place追加
// C: Condition Place追加
// R: Result Place追加
// T: Transition追加
// Ctrl+Z: Undo
// Ctrl+Y: Redo
// Ctrl+C: Copy
// Ctrl+V: Paste
// Ctrl+S: Save
```

8.2 キャンバスクリック時のノード追加

```
const handlePaneClick = useCallback((event: React.MouseEvent) => {
  if (selectedTool === 'workflow') {
    // ワークフロー追加モード
    const position = screenToFlowPosition({
      x: event.clientX,
      y: event.clientY,
    });
    setPendingClickPosition(position);
    openWorkflowDialogInternal();
    return;
  }

  // 押されているキーをチェック
  let nodeType = null;
  if (pressedKeys.has('m')) {
    nodeType = 'material';
  } else if (pressedKeys.has('c')) {
    nodeType = 'condition';
  } else if (pressedKeys.has('r')) {
    nodeType = 'result';
  } else if (pressedKeys.has('t')) {
    nodeType = 'transition';
  }

  if (!nodeType) return;

  const position = screenToFlowPosition({
```

```
      x: event.clientX,
      y: event.clientY,
    });

    const newNode: Node<NodeData> = {
      id: `${nodeType}-${Date.now()}`,
      type: nodeType === 'transition' ? 'transition' : `${nodeType}Template`,
      position,
      data: {
        id: `${nodeType}-${Date.now()}`,
        name: '',
        description: '',
        placeType: nodeType,
        nodeType: (nodeType === 'transition' ? 'transition' : 'place') as NodeType,
      },
    };

    setNodes((nds) => {
      const newNodes = [...nds, newNode];
      pushState({ nodes: newNodes, edges });
      return newNodes;
    });
  }, [pressedKeys, edges, pushState, setNodes, screenToFlowPosition, selectedTool]);
```

---

9. API連携

9.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	説明
1	バリデーションAPI	POST	`/api/v1/maiml/validate`	MAIML XMLをバリデーション
2	自動補正API	POST	`/api/v1/maiml/auto-correct-realtime`	バリデーションエラーを自動補正
3	MAIML保存API	POST	`/api/v1/maiml/save`	S3にMAIML保存
4	ストレージファイル一覧取得	GET	`/api/v1/storage/files/:orgId`	組織のファイル一覧取得
5	ストレージファイル内容取得	GET	`/api/v1/storage/file-content`	S3ファイルの内容取得

9.2 バリデーションAPI呼び出し

```
// Base64エンコード
const base64Data = btoa(unescape(encodeURIComponent(maimlXml)));
```

```
// API呼び出し
const response = await maimlService.validateMaiml(base64Data);

if (response.success) {
  const result = response.data as AnalysisResult;
  setValidationResult(result);
}
```

9.3 自動補正API呼び出し

```
const response = await maimlService.autoCorrectMaimlRealtime(
  base64Data,
  selectedErrors, // 選択されたエラーのインデックス配列
  currentOrgId
);

if (response.success && response.data.correctedContent) {
  const correctedMaiml = atob(response.data.correctedContent);
}
```

---

10. エラーハンドリング

10.1 エラー分類

```
// 致命的エラー（ペトリネット表示不可）
const criticalTypes = ['syntax_error', 'invalid_structure'];

// 非致命的エラー（表示可能）
const nonCritical = validationResult.errors.filter(err =>
  !criticalTypes.includes(err.type)
);
```

10.2 エラー可視化

```
useEffect(() => {
  if (nonCriticalErrors.length > 0) {
    const { nodeIds, edgeIds } = extractAffectedElements(nonCriticalErrors);

    // ノードにエラーフラグを追加
    setNodes(prevNodes =>
      prevNodes.map(node => {
```

```
        const hasError = nodeIds.includes(node.id);
        return {
          ...node,
          data: {
            ...node.data,
            hasError: hasError,
            errors: nodeErrors
          },
          className: hasError ? 'error-node' : ''
        };
      })
    );

    // エッジにエラースタイルを追加
    setEdges(prevEdges =>
      prevEdges.map(edge => {
        const hasError = edgeIds.includes(edge.id);
        return {
          ...edge,
          className: hasError ? 'error-edge' : '',
          style: hasError ? {
            ...edge.style,
            stroke: '#f44336',
            strokeWidth: 3
          } : edge.style
        };
      })
    );
  }
}, [nonCriticalErrors, setNodes, setEdges]);
```

10.3 エラーケース

エラーケース	対処方法
致命的エラー（syntax_error等）	ペトリネット表示を停止し、エラーメッセージを表示
非致命的エラー（duplicate_id等）	警告バナー表示、該当ノード/エッジを赤色で強調
初期化エラー	エラーメッセージ表示、onErrorコールバック呼び出し
バリデーションエラー	アラート表示
保存エラー	アラート表示、ダイアログは開いたまま

---

11. 実装ファイル

11.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/PetriNetEditorInner.tsx`

11.2 関連ファイル

- **型定義**: `frontend/src/types/editor.ts`, `frontend/src/types/maiml-unified.ts`
- **コンテキスト**: `frontend/src/contexts/TemplateContext.tsx`
- **カスタムフック**:
  - `frontend/src/components/editorv2/hooks/useHistory.ts`
  - `frontend/src/components/editorv2/hooks/useWorkflow.ts`
  - `frontend/src/components/editorv2/hooks/useDraftManager.ts`
  - `frontend/src/components/editorv2/hooks/useKeyboardShortcuts.ts`
  - `frontend/src/components/editorv2/hooks/useEditorActions.ts`
- **ユーティリティ**:
  - `frontend/src/utils/maimlParserTyped.ts`
  - `frontend/src/utils/reactFlowConverter.ts`
  - `frontend/src/components/editorv2/utils/converters/reactFlowToMaiml.ts`
  - `frontend/src/components/editorv2/utils/converters/multiMethodConverter.ts`
  - `frontend/src/utils/documentInitializer.ts`
- **APIサービス**: `frontend/src/services/maimlApiService.ts`

11.3 子コンポーネント

- `<EditorToolbar />` - ツールバー
- `<MethodTabPanel />` - メソッドタブ
- `<PropertyPanel />` - プロパティパネル
- `<SaveConfirmDialog />` - 保存確認ダイアログ
- `<TemplateEditorDialog />` - テンプレート編集ダイアログ
- `<DocumentEditDialog />` - Document編集ダイアログ
- `<ValidationPanel />` - バリデーションパネル
- `<WorkflowSelectorDialog />` - ワークフロー選択ダイアログ
- `<DraftManager />` - Draft管理ダイアログ
- `<TemplateList />` - テンプレート一覧（templatesモード時）
- ノードコンポーネント:
  - `<MaterialTemplateNode />`

- `<ConditionTemplateNode />`
- `<ResultTemplateNode />`
- `<TransitionNode />`

---

12. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline